# P2

# LFG Dungeon Queue Synchronization

C++17

Ferrer, Lance

# Problem and Requirements

## One party

1 tank + 1 healer + 3 DPS

## Execution rules

At most n active dungeons
All players arrive together
No deadlock or eligible-party starvation

| Input | Meaning |
| --- | --- |
| n | Concurrent instance limit |
| t, h, d | Tank, healer, and DPS counts |
| t1, t2 | Inclusive duration bounds (seconds) |

n >= 1, player counts >= 0
0 <= t1 <= t2 <= 15

# Build and Execution

From the project root in Windows Command Prompt

```
g++ -std=c++17 -pthread -Iinclude ^
    src/main.cpp src/simulation.cpp ^
    -o dungeon_queue.exe
```

```
dungeon_queue.exe

2 4 3 10 1 1
```

Use a modern GCC with C++17 support.
Compile both source files.
The executable goes in the project root.

Input order: n t h d t1 t2





# Project Structure

| File or group | Role |
| --- | --- |
| `src/main.cpp` | Console input, simulation call, summary, errors |
| `src/simulation.cpp` | Party queue, workers, status, and output |
| `include/simulation.hpp` | Shared types and simulation interface |
| `include/counting_semaphore.hpp` | Custom C++17 counting semaphore |
| `tests/` | Simulation, semaphore, and CLI checks |
| `CMakeLists.txt / README.md` | Build setup and documentation |

# Party Formation

```
party_count = min(t, h, d / 3)
```

| Party | Tank | Healer | DPS |
|---|---|---|---|
| 1 | T1 | H1 | D1, D2, D3 |
| 2 | T2 | H2 | D4, D5, D6 |
| 3 | T3 | H3 | D7, D8, D9 |

Example: t = 4, h = 3, d = 10

3 complete parties. Leftovers: 1 tank, 0 healers, 1 DPS.





# Worker Execution

## Before workers start

Form complete parties
Append n shutdown markers
Publish one permit per queue item

## One worker per instance

The n workers enforce the limit.
Each worker runs one party at a time.

```
1. Acquire a job permit
2. Remove the FIFO job
3. Exit if it is a marker
4. Mark instance active
5. Sleep for dungeon duration
6. Record completion
7. Repeat
```

Dungeon sleep holds no application mutex.

# Synchronization Mechanisms

| Mechanism | Responsibility |
| --- | --- |
| CountingSemaphore | One permit per queued party or shutdown marker |
| std::condition_variable | Block acquire() until a permit is available |
| queue_mutex | Make queue removal exclusive |
| status_mutex | Keep state, statistics, and output coherent |
| RAII lock guards | Release mutexes when a scope exits |
| std::thread::join() | Wait for all workers before final reporting |





# Custom Counting Semaphore

## acquire()

```
void acquire() {
    std::unique_lock<std::mutex>
        lock(mutex_);
    available_.wait(lock, [this] {
        return permits_ != 0;
    });
    --permits_;
}
```

## release() excerpt

```
{
    std::lock_guard<std::mutex>
        lock(mutex_);
    // ... overflow check ...
    permits_ += count;
}
if (count == 1) {
    available_.notify_one();
} else {
    available_.notify_all();
}
```

Predicate waiting handles spurious wakeups.
Published permits remain available until acquired.





# Protected Critical Sections

## Queue removal

```
{
    std::lock_guard<std::mutex>
        lock(queue_mutex);
    party = jobs.front();
    jobs.pop();
}
```

## State and output

```
{
    std::lock_guard<std::mutex>
        lock(status_mutex);
    summary.instances[
        static_cast<std::size_t>(index)
    ].active_party = party;
    observe(Event::started,
            index + 1, party);
}
```

```
sleep(std::chrono::seconds(party->duration_seconds));
```

Separate lock scopes. Automatic release. No mutex during dungeon sleep.





# Possible Deadlocks

## Hypothetical fault: opposite lock order

| Worker A | Worker B |
| --- | --- |
| Holds queue_mutex | Holds status_mutex |
| Waits for status_mutex | Waits for queue_mutex |

Each worker waits for a lock held by the other.

## Hypothetical fault: incomplete shutdown

No signaled shutdown markers: workers remain in acquire().
Main then waits indefinitely in join().

# Deadlock Prevention

### No circular lock dependency

Queue and status locks have separate scopes.
Workers acquire permits and sleep outside those scopes.

### No unfinished party waits

Main forms every complete party before starting workers.

### A complete shutdown protocol

Every queue item has a permit.
Party jobs come before n exit markers, then main joins all workers.

# Starvation and Fairness

## Risk

An eligible participant waits forever
while others progress.
Unfair access can repeatedly bypass it.

## Party fairness

FIFO role IDs and FIFO jobs
Finite input, bounded dungeon runs
No later arrivals bypass queued parties

## Fairness assumptions

Runnable threads eventually run and synchronization makes progress.
FIFO waiter wakeups and equal instance workloads are not guaranteed.

Unmatched roles remain unassigned: 1 tank + 1 DPS cannot form a party.

# Shutdown and Error Handling

```
for (int index = 0;
     index < config.instance_count; ++index) {
    jobs.push(std::nullopt);
}
available_jobs.release(jobs.size());
```

### Normal completion

One marker stops one worker.
Main joins all workers before reporting.

### Exceptional cleanup

Atomic cancellation stops new claims.
Main joins workers, then rethrows.
Partial thread creation also joins safely.

Jobs and markers are ready before worker creation.





# Example Run

Input: 2 4 3 10 1 1

```
Instance 1: active | Party 1 | Tank T1 | Healer H1
  | DPS D1, D2, D3 | Duration 1 s
Instance 2: active | Party 2 | Tank T2 | Healer H2
  | DPS D4, D5, D6 | Duration 1 s
```

| Instance | Parties served | Total time served |
|---|---|---|
| 1 | 2 | 2 s |
| 2 | 1 | 1 s |

Both instances ended empty. Instance assignments may vary.

Total parties completed: 3

Unassigned: 1 tank, 0 healers, 1 DPS





# Verification and Conclusions

## Verified behavior

Correct parties and unique player IDs
FIFO selection and accurate leftovers
Concurrent runs within the n limit
Coherent statistics and clean termination

## Observed test results

7 C++ test groups passed
8 CLI checks passed
Real example: 3 completed parties

## Design conclusions

Workers bound capacity. The semaphore accounts for jobs.
Short, separate lock scopes protect shared state.
FIFO finite work and exit markers support eventual completion.